

Картофель

- **Ограничение по времени:** 10 минут
- **Среда:** один GPU (≈16 GB VRAM), без интернета
- **Размер решения:** `solution.ipynb` ≤ 1 MB
- **Хранилище:** 5 GB

Задача

Ваш друг предлагает сыграть в игру на угадывание. Он, выступая в роли судьи, выбирает одно скрытое слово из фиксированного словаря, а вы должны найти его не более чем за 30 ходов. На каждом ходу судья сравнивает два слова и сообщает, какое из них семантически ближе к скрытому слову. Каждая игра начинается с фиксированной пары `lamp` vs `potato`, поскольку это две из любимых вещей вашего друга. Затем ваша программа предлагает одно новое слово. Победитель сравнения сохраняется и сравнивается с вашим следующим предложением. Вы выигрываете игру в тот момент, когда предлагаете скрытое слово в точности. Регистр не учитывается. Каждое предлагаемое вами слово должно находиться в `dataset/vocabulary.json`.

Полный пример с протоколом и загрузкой данных приведён в `solution.ipynb`. Вы можете изменить класс `PublicEmbeddingPlayer`. Ваша программа инициализируется один раз и играет все игры за один запуск; протокол создаёт новый экземпляр `PublicEmbeddingPlayer` в начале каждой игры.

Судья

Ваша программа отправляет судье один объект JSON, а судья отвечает другим одним объектом JSON.

Разобранный пример, в котором скрытое слово показано только для объяснения протокола:

```
Hidden word: shovel          Fixed opening: lamp vs potato

<- {"turn": 1, "winner_word": "potato", "verdict": "second", "word1": "lamp", "word2": "potato"}
-> {"new_word": "rock"}
<- {"turn": 2, "winner_word": "rock", "verdict": "second", "word1": "potato", "word2": "rock"}
-> {"new_word": "hammer"}
<- {"turn": 3, "winner_word": "hammer", "verdict": "second", "word1": "rock", "word2": "hammer"}
-> {"new_word": "shovel"}
-> {"status": "win"}          <- matches: game won
```

Ходы нумеруются от 1 до 30.

Возможные значения `verdict`: `first`, означающее, что `word1` ближе, `second`, означающее, что `word2` ближе, или `same`, означающее, что оба слова одинаково близки к скрытому слову.

`winner_word` — это слово, сохраняемое для следующего сравнения. При вердикте `same` остаётся первое слово.

Датасет

Общее для всех разбиений:

- `dataset/vocabulary.json` — 1602 уникальных слова в нижнем регистре. Скрытое слово всегда является одним из них.
- `dataset/public_embeddings.npy` — float32, матрица размера (1602, 2560). Строка `i` соответствует слову `i` в словаре. Это публичные эмбединги; судья использует другие представления, они вам не доступны.

Сабсеты представляют собой множества скрытых слов:

Сабсет	Слова	Ответы	Использование
<code>test_public</code>	120	<code>dataset/test_public.json</code>	запускать своё решение и самостоятельно оценивать результат
<code>test_leaderboard_a</code>	120	скрыты	текущая таблица лидеров
<code>test_leaderboard_b</code>	120	скрыты	итоговый рейтинг

Сабсета `train` нет — по размеченным строкам ничего не обучается.

Предоставленные модели

Вместе с задачей вам доступны две предобученные модели эмбедингов, которые можно использовать:

```
models/Qwen/Qwen3-Embedding-0.6B
[Qwen Embedding model card] (https://yastatic.net/s3/contest/ioai/3/01_Qwen_Qwen3-Embedding-0.6B_MODEL_CARD.html)
models/BAAI/bge-m3
[bge-m3 model card] (https://yastatic.net/s3/contest/ioai/3/02_BAAI_bge-m3_MODEL_CARD.html)
```

Обе модели необходимо загружать из соответствующего локального пути; идентификатор Hugging Face Hub, такой как "BAAI/bge-m3", запускает скачивание и приводит к ошибке, поскольку проверка выполняется без интернета. Каждый каталог содержит запускаемый `example.py`, демонстрирующий вызов без интернета.

Доступные библиотеки: `numpy`, `torch`, `sentence-transformers`. Скачивание и установка других пакетов и весов недоступна.

Вывод

Отсутствует. Это интерактивная задача: ваше решение не записывает файл с ответом, а взаимодействует с судьёй через `stdin/stdout`, как описано выше.

Метрика

Игра, в которой слово найдено на ходу t , приносит $1.0 - 0.02 \times \max(0, t - 10)$; игра, не решённая за 30 ходов, приносит 0. Таким образом, решения за ходы 1–10 приносят 1.00, решение на 20 ходу приносит 0.80, а на 30 ходу приносит 0.60.

Ваш балл за задачу равен среднему баллу за игру $\times 100$ и находится между 0.00 и 100.00.

Ограничение в 10 минут представляет собой единый бюджет, включающий запуск, подготовку и все 120 игр в тестовом наборе.

Как отправить решение

1. Откройте `solution.ipynb`, отредактируйте `PublicEmbeddingPlayer` и выполните все ячейки, чтобы убедиться, что всё работает.
2. При желании проверьте решение локально: `python local_test.py solution.ipynb --limit 5`. Локальный судья использует *публичные* эмбединги, поэтому полученный балл служит лишь ориентиром.
3. Сохраните `solution.ipynb`.
4. Откройте вкладку Git на левой боковой панели JupyterLab.
5. Добавьте `solution.ipynb` в индекс (значок + рядом с ним).
6. Введите сообщение коммита и нажмите Commit.
7. Нажмите значок облака со стрелкой вверх, чтобы выполнить отправку.
8. Вернитесь на эту страницу конкурса и нажмите Submit, при этом сообщение коммита должно совпадать с указанным вами.

Отправьте ровно один файл с именем `solution.ipynb`, включающий всю необходимую подготовку и инференс.